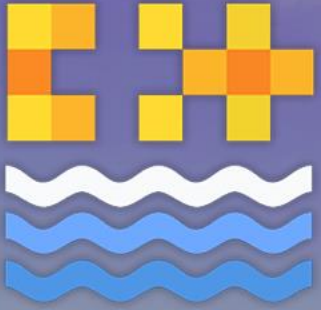




Being Friendly to Your Hardware

Ignas Bagdonas

2024

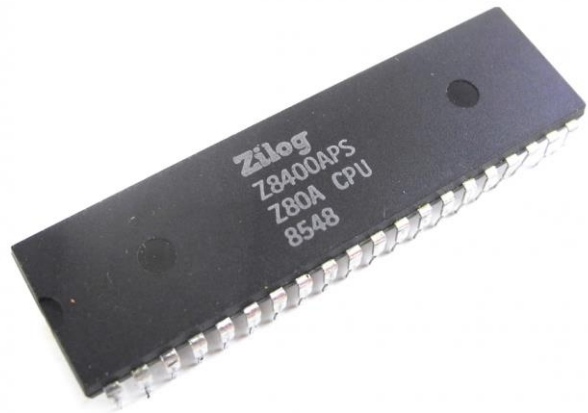
Being Friendly to Your Hardware

Performance Engineering

A gentle introduction to hardware for software engineers

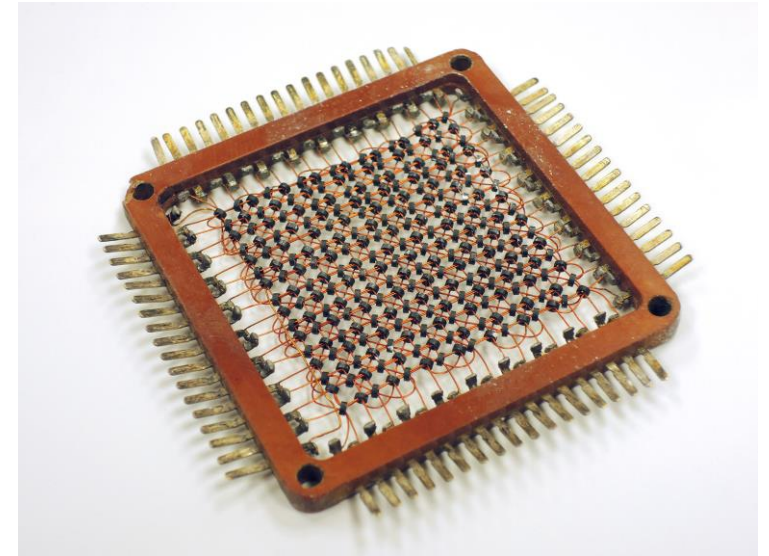
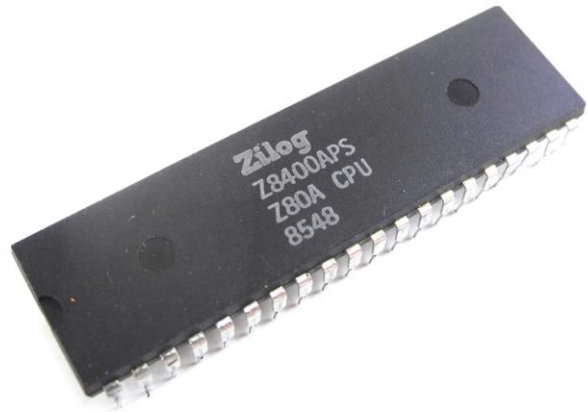
Computer system

Processor + Interconnect + Memory



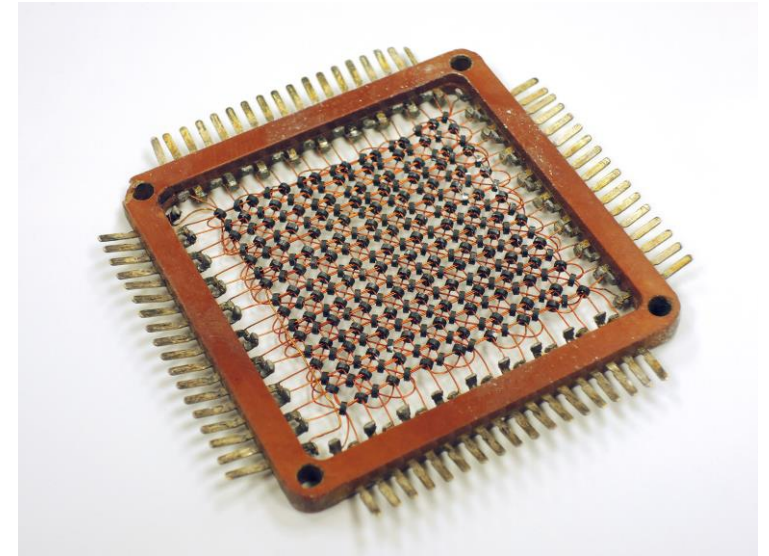
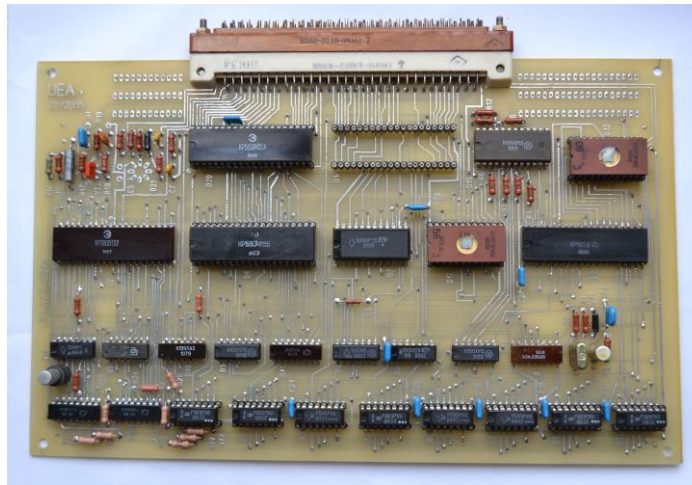
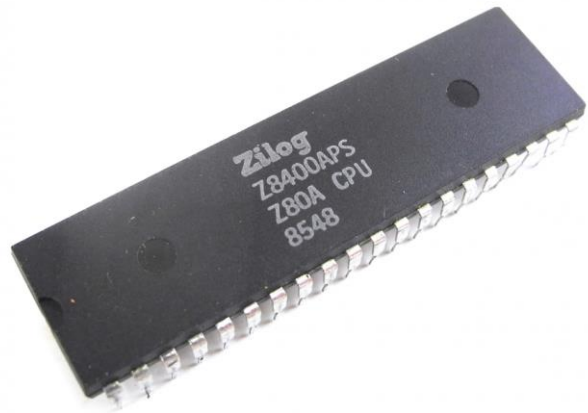
Computer system

Processor + Interconnect + Memory



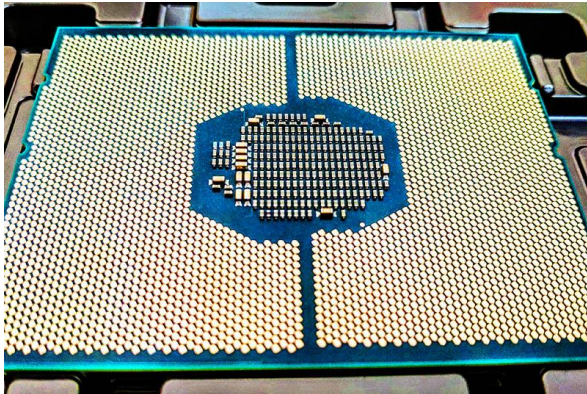
Computer system

Processor + Interconnect + Memory



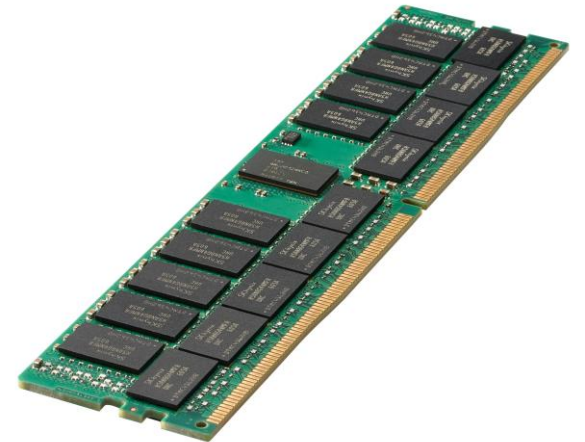
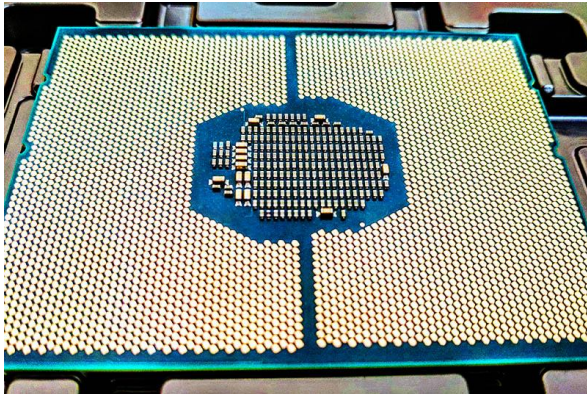
Computer system

Processor core + Interconnect + Memory



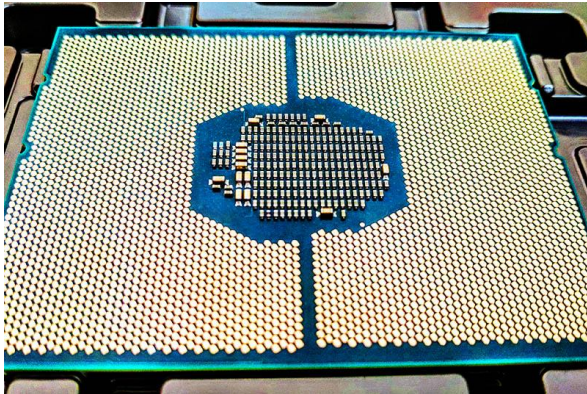
Computer system

Processor core + Interconnect + Memory



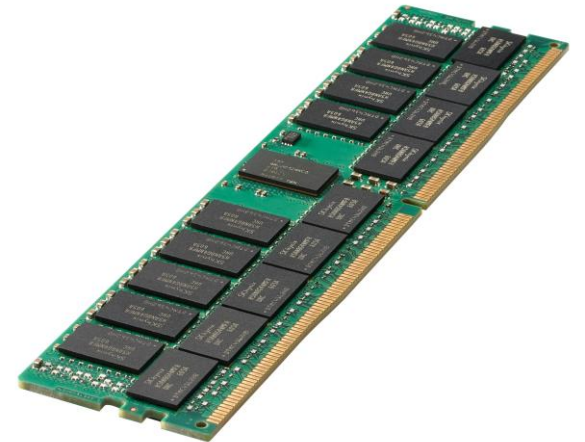
Computer system

Processor core + Interconnect + Memory



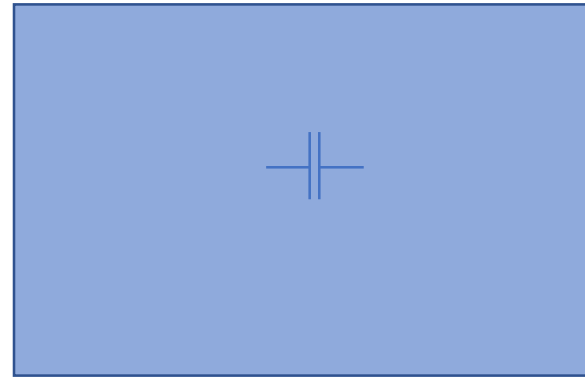
Complex and scary

And mostly out of
scope anyway



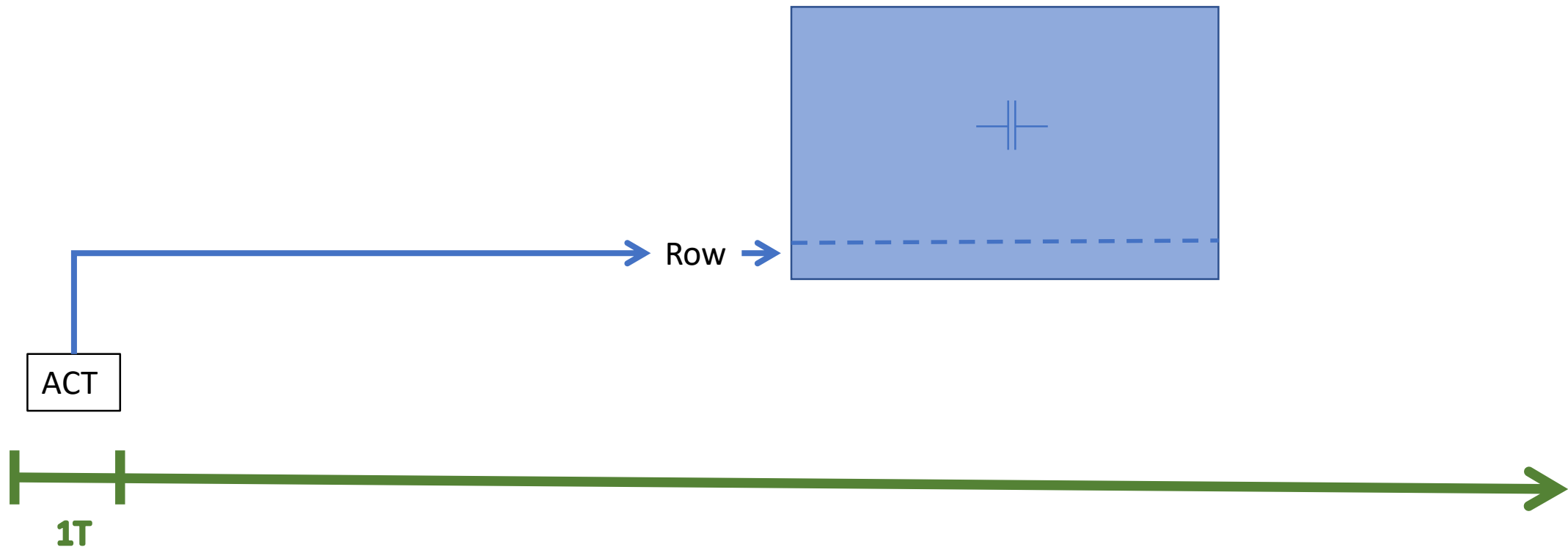
Memory

- Addressable capacitor array
- Short term nonvolatile
- Speeds – capacitor array vs bus interface



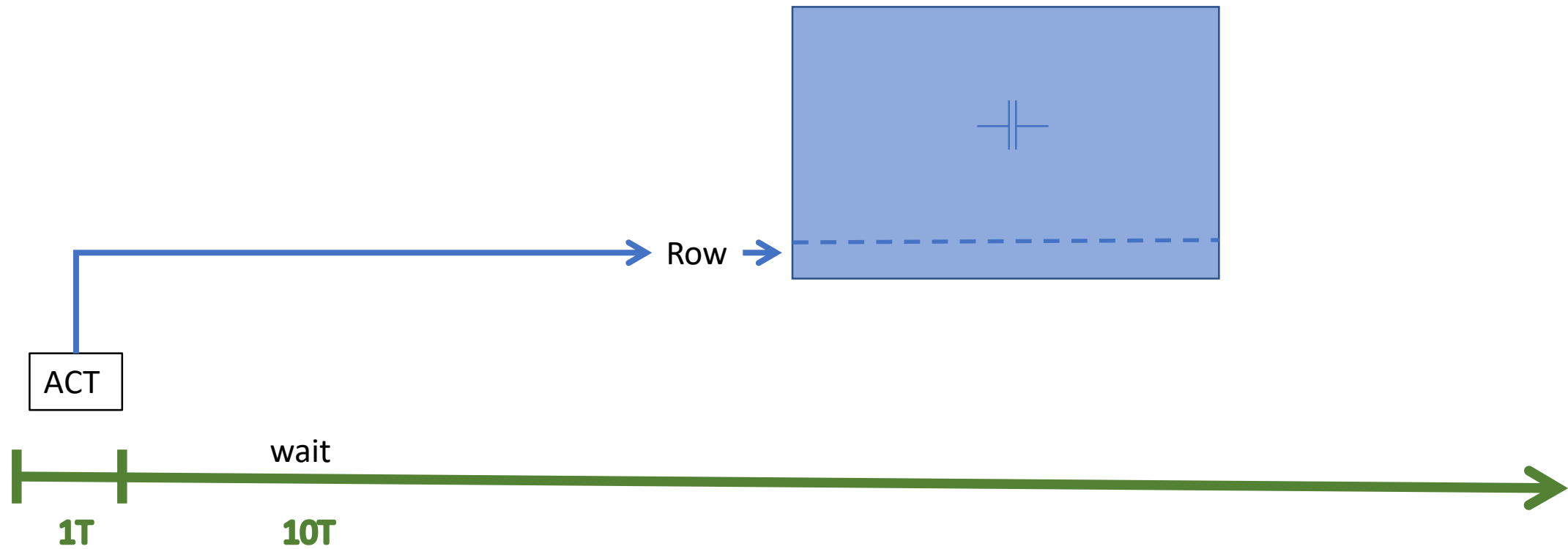
Memory

- Open a row
- Send part of address - one command



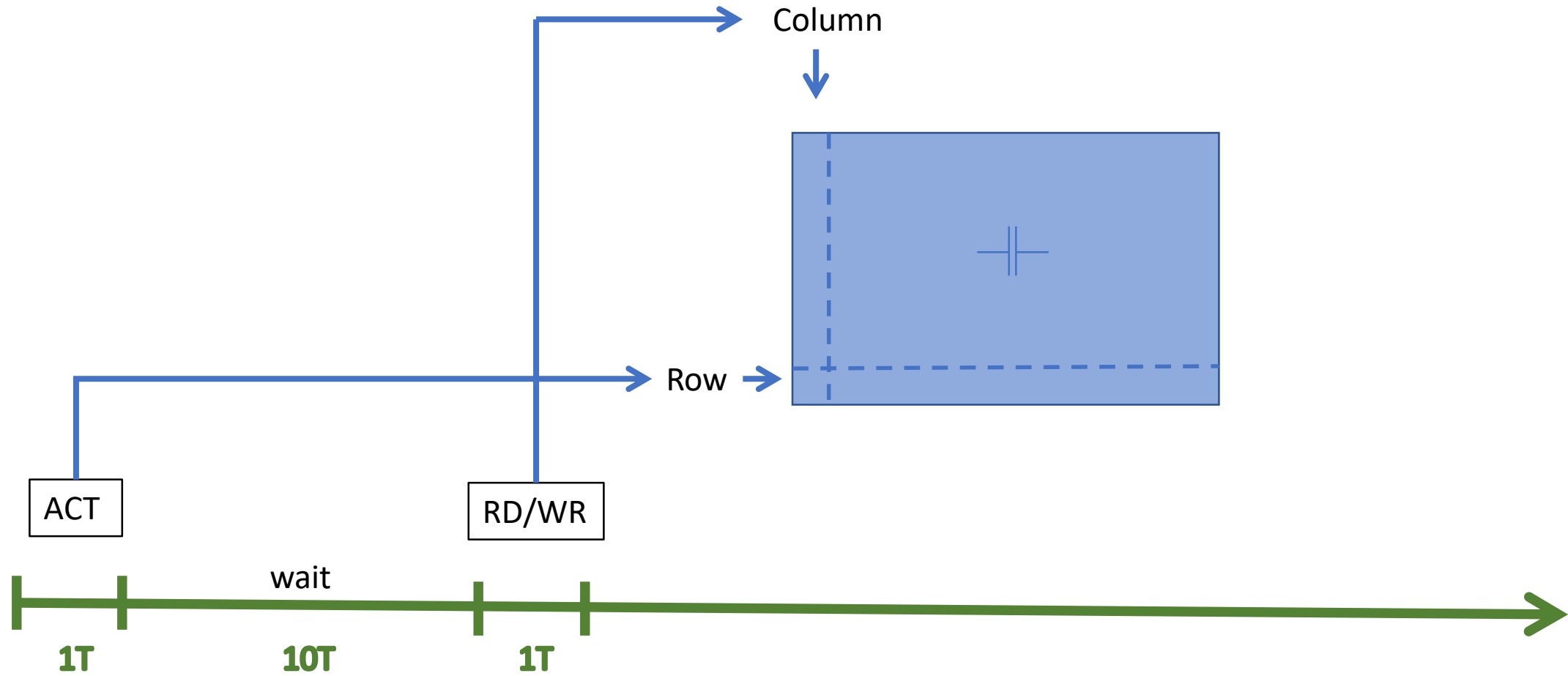
Memory

- Wait for a portion of array to get selected.



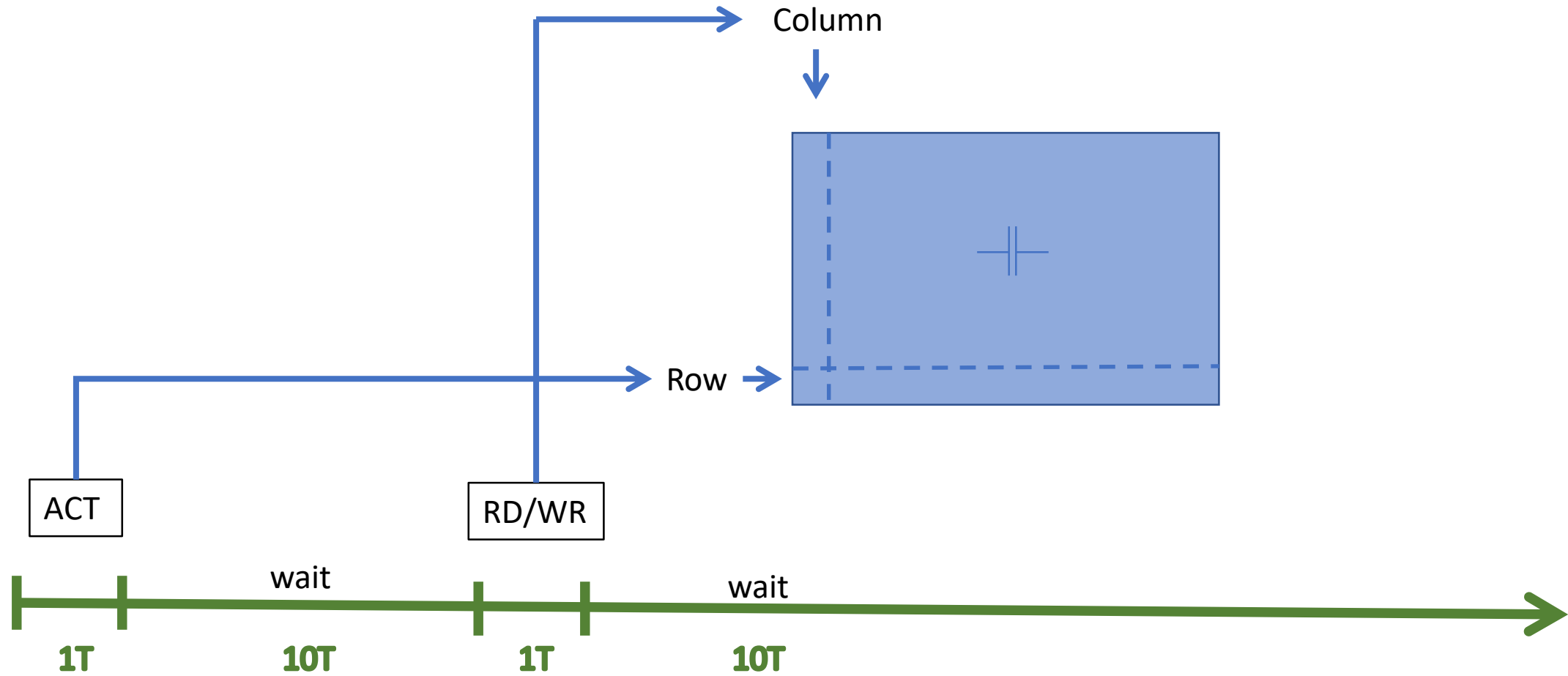
Memory

- Send column address and actual memory operation – another command



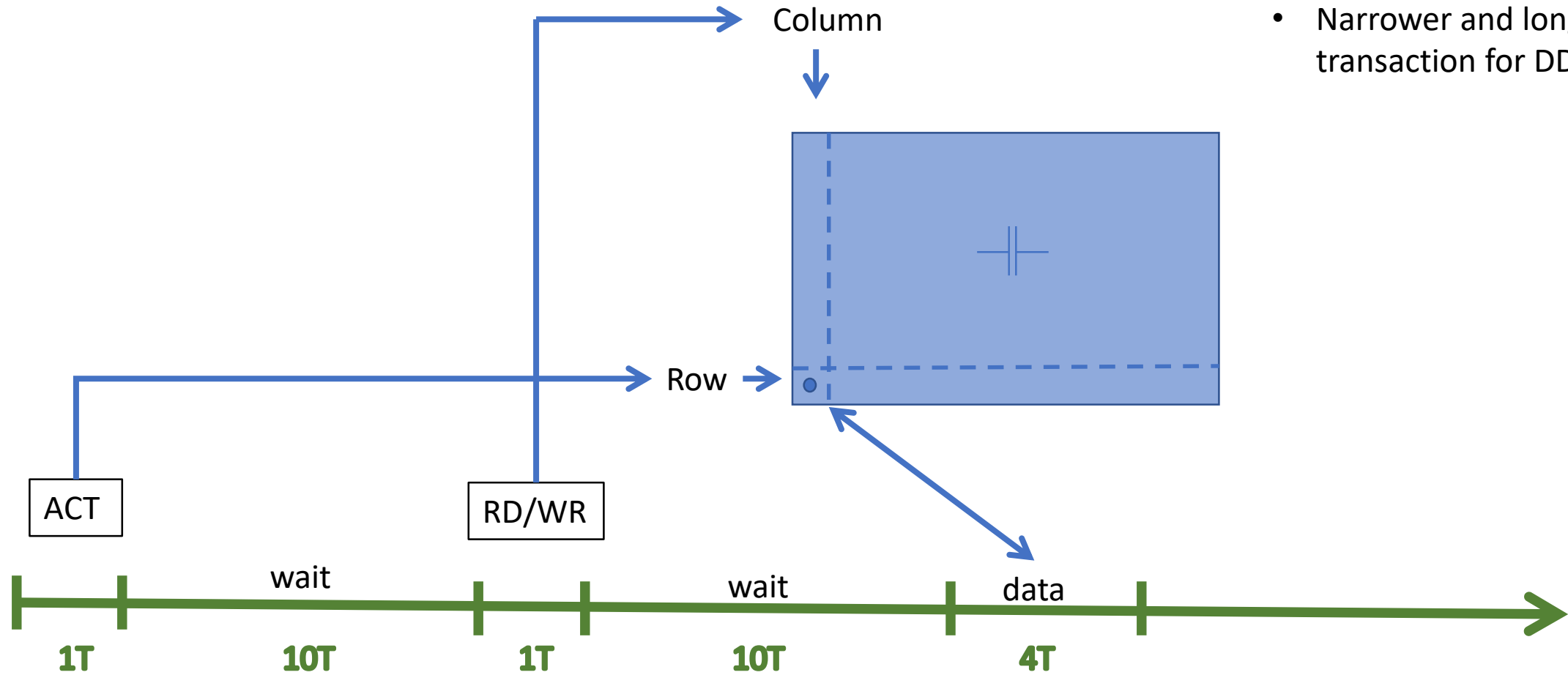
Memory

- Wait for a portion of row to get selected and capacitors sensed.



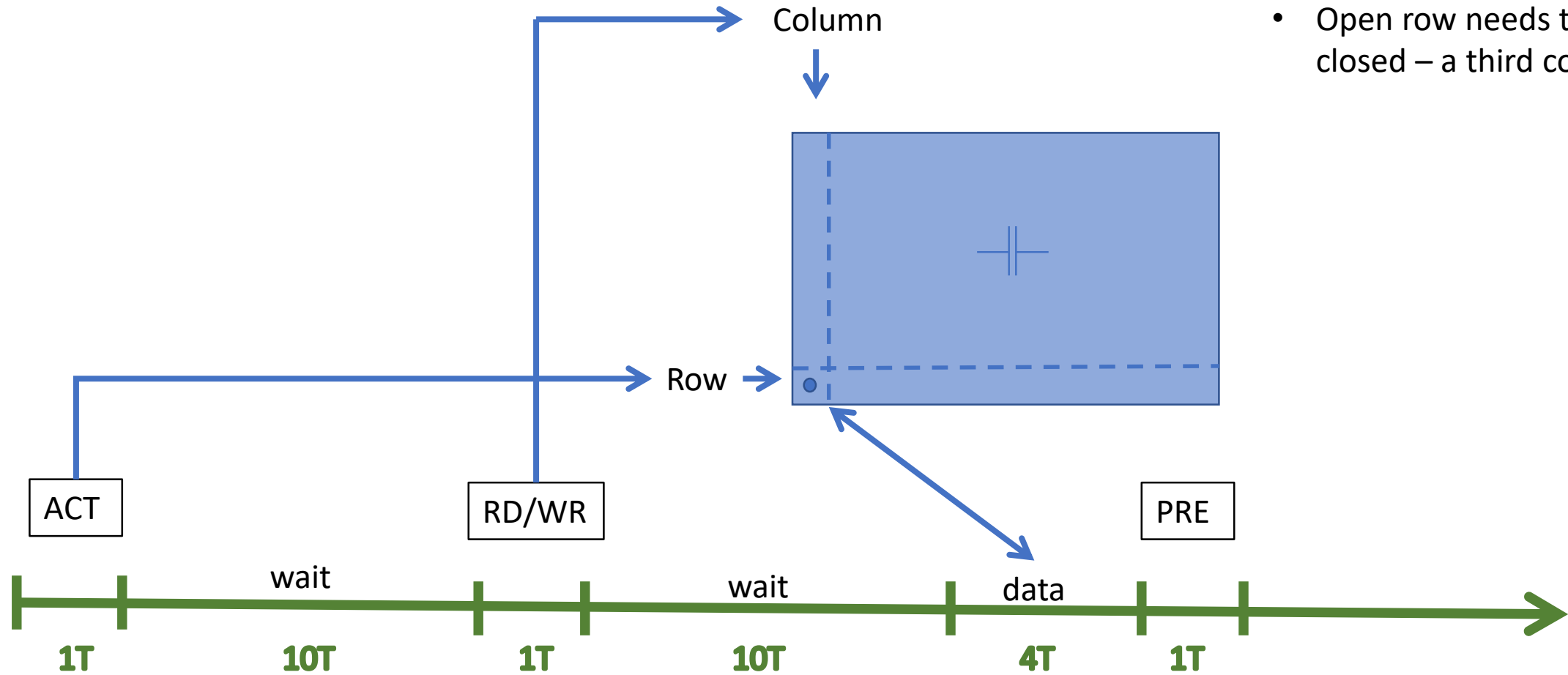
Memory

- Actual data transfer.
- Wider and shorter transaction for DDR4
- Narrower and longer transaction for DDR5



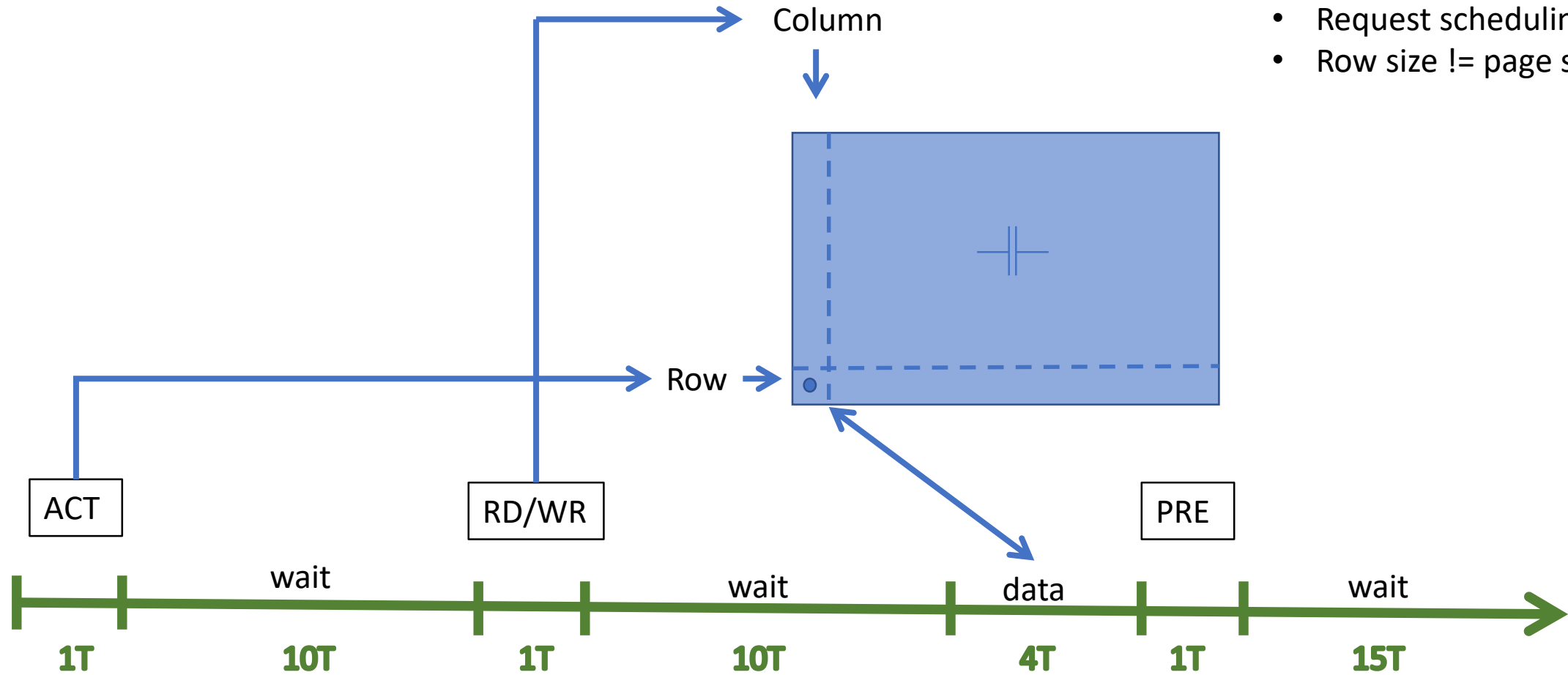
Memory

- Capacitor array access is destructive
- Even for a read
- Open row needs to be closed – a third command



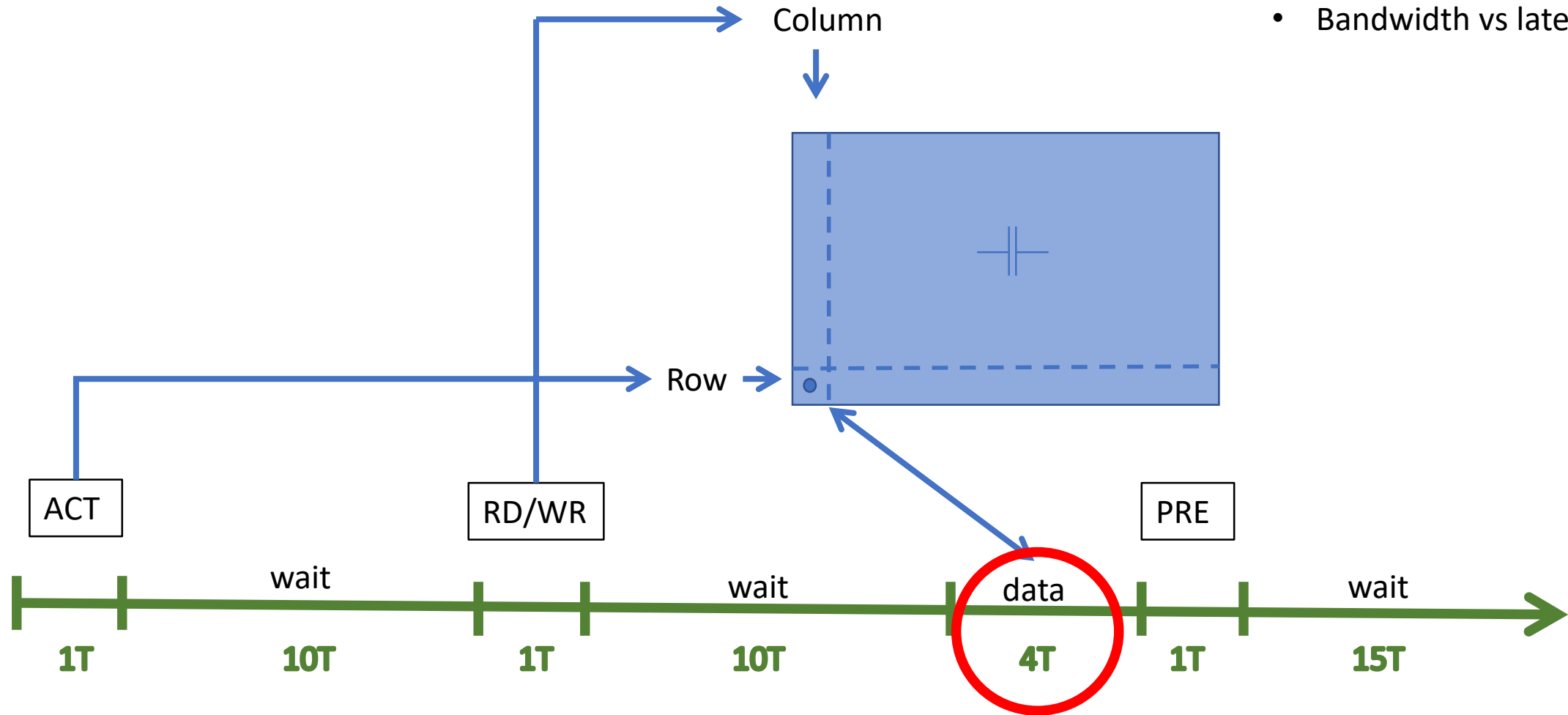
Memory

- One row active at a time
- All rows need to be refreshed periodically
- Request scheduling
- Row size $\neq$ page size



Memory

- Usable utilization is quite far away from theoretical limit
- Bandwidth vs latency



Memory

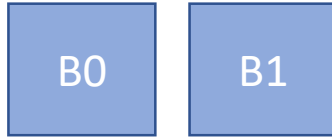
- DDR architecture – it is only a fraction of bus transaction that is in fact DDR
- Row addressing
- Column addressing
- Generation to generation capacitor array operates at mostly the same frequency, bus transfer speed in fact increases
- Multiple memory modules (sockets)
- Page size – CPU vs memory
- Address mapping

Memory



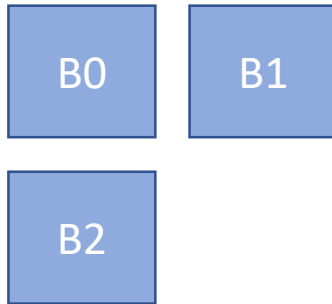
- One row open per bank
- Logical partitioning

Memory



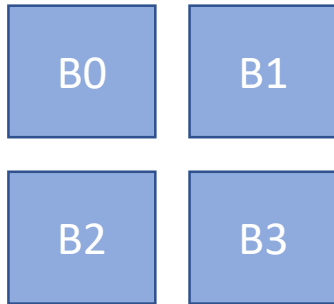
- One row open per bank
- Logical partitioning

Memory



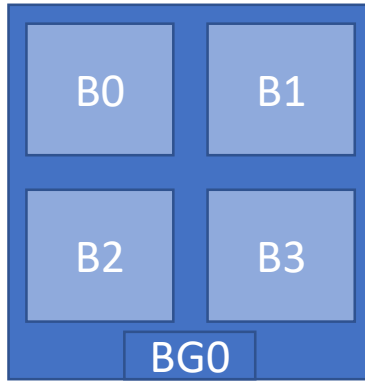
- One row open per bank
- Logical partitioning

Memory



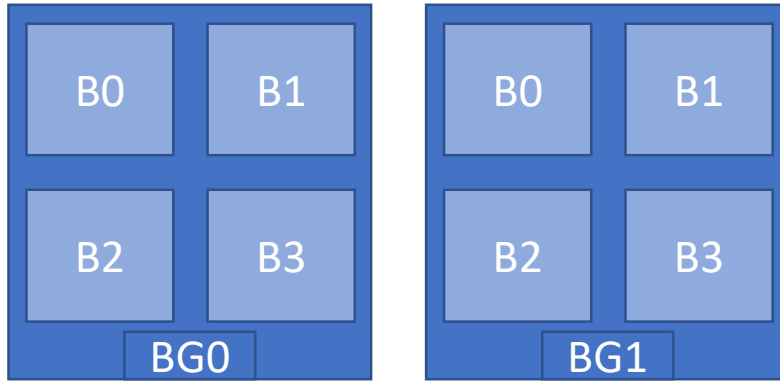
- One row open per bank
- Logical partitioning

Memory



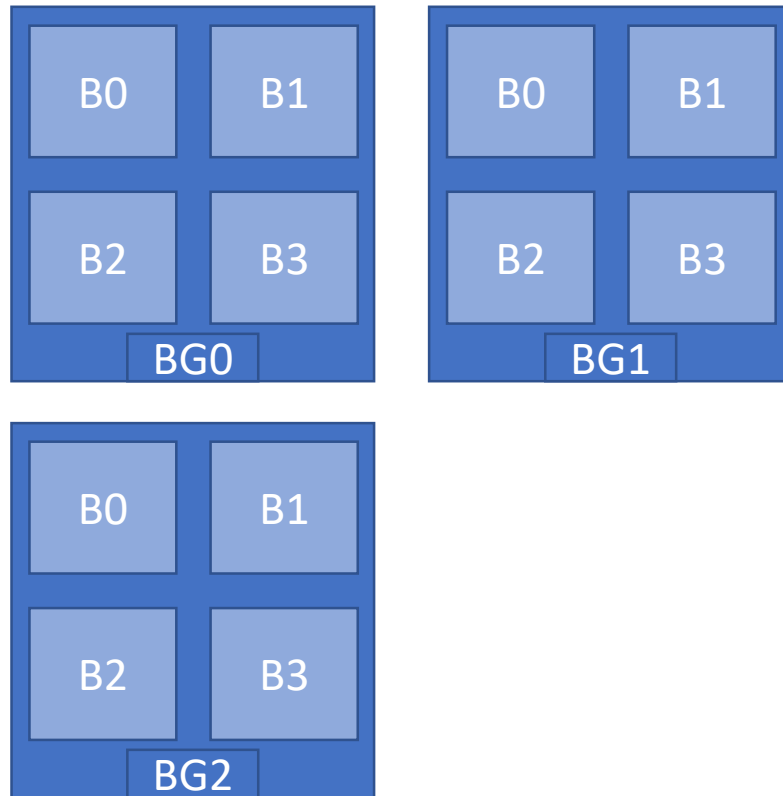
- Bank group – a grouping of logically addressable banks
- Multiple open rows per bank group
- Multiple outstanding commands in progress

Memory



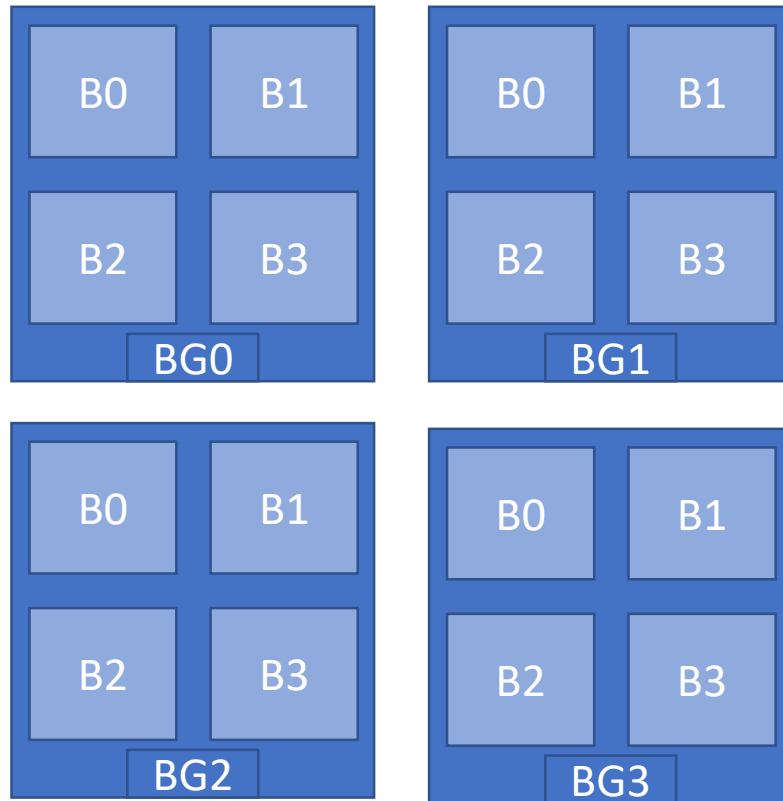
- Bank group – a grouping of logically addressable banks
- Multiple open rows per bank group
- Multiple outstanding commands in progress

Memory



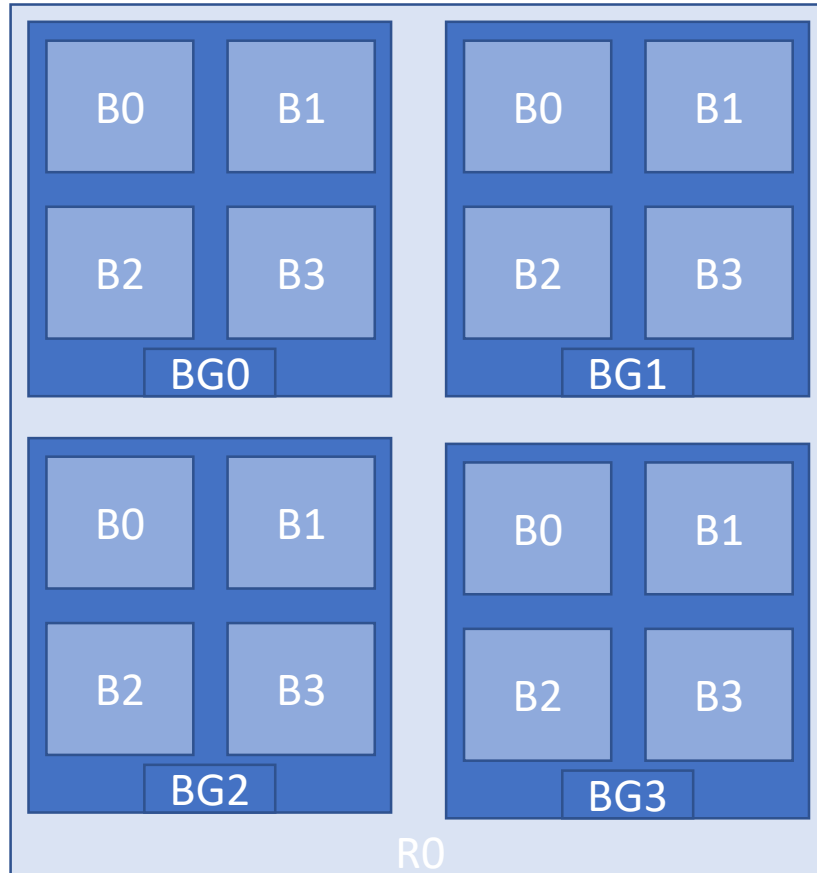
- Bank group – a grouping of logically addressable banks
- Multiple open rows per bank group
- Multiple outstanding commands in progress

Memory



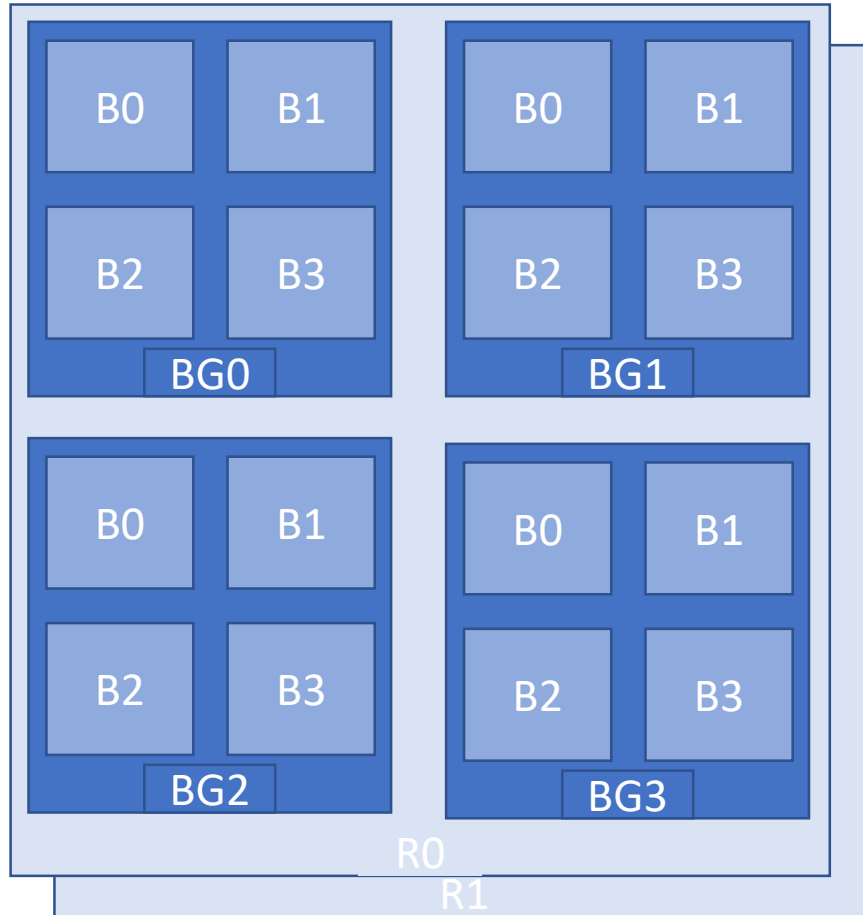
- Bank group – a grouping of logically addressable banks
- Multiple open rows per bank group
- Multiple outstanding commands in progress

Memory



- Multiple bank groups for a rank
- A rank is just a CS line from the electrical domain perspective
- It is part of addressing scheme too

Memory



- Multiple bank groups for a rank
- A rank is just a CS line from the electrical domain perspective
- It is part of addressing scheme too
- Multiple ranks physically equal to multiple separate groups of memory components assembled on the same PCB
- MDP and vendor marketing names for multilayer components

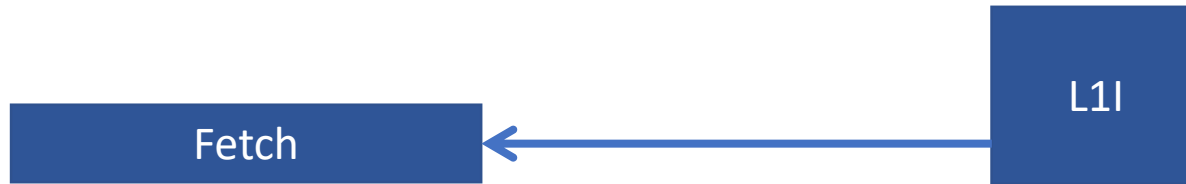
Memory

- What about HBM?
 - Not 1024 bits, 8 x 128 instead.
 - Bandwidth will only be high when there is sufficient stream of commands inflight
- What about remote memory?
 - CXL and vendor proprietary interconnects
 - Coherency is expensive
 - Single physical address space is expensive
 - Single logical cache coherent address space is very expensive
- What about nonvolatile memory?
 - NVDIMM-* variants, vendor specifics.
 - Useful for debugging
- System address mapping design and control

Memory

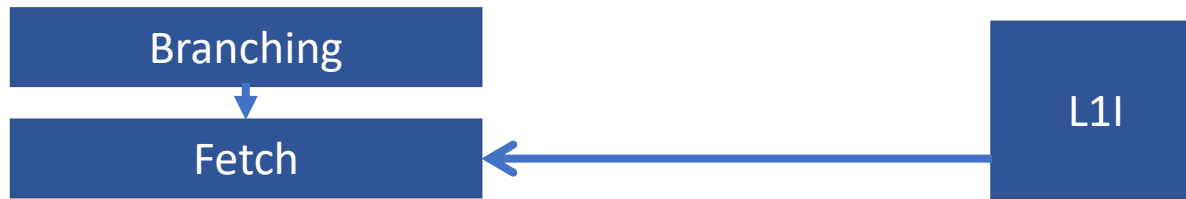
- Memory system is in the uncore
- Cores act as clients
- Remote socket cores act as clients
- Distributed cache coherence is hard and expensive
- Writing implies reading first in most of cases
- Reading has a load on cache hierarchy
- Nontemporal writes may help in some cases
- Cacheability parameters can be tuned, this is highly platform dependent though.

Instruction fetch



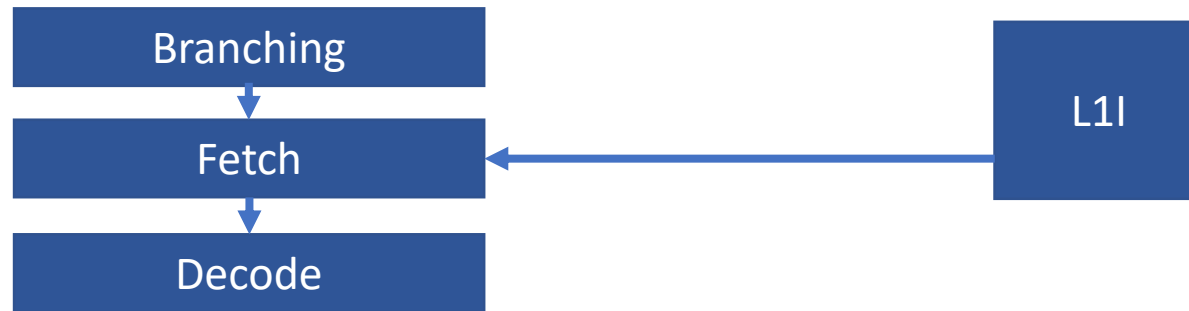
- Gets next block of (partial) instructions
- Linear fetch
- Incoming branch
- Instruction alignment
- Instruction fusing

Branch prediction



- Governs fetching of next instruction blocks
- A set of tables
- Branch history
- Branch site
- Branch target
- Repetitive patterns
- Really complex
- High cost of error

Instruction decoding

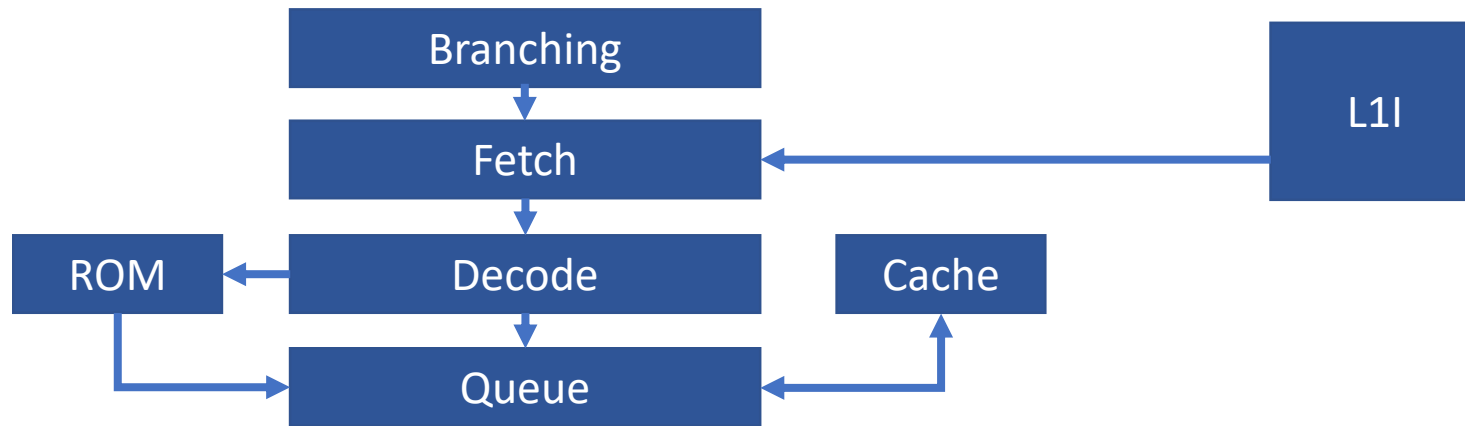


- Multiple instructions get decoded in parallel
- Even for variable length encoding
- Not that complex in HW domain
- Fusion

Instruction input

- Has to be logically sequential
- Can be physically parallel
- HW is inherently parallel
- Myth: decoding variable length instructions is:
 - Complex
 - Serial
 - Slow
- Fetch block size
- Linear fetch vs incoming branch

Instruction decoding



- Decoded operations may get cached
- There is a library of operations for complex instructions and events
- Decoded operations may get fused

Code density

```
uint64_t v = 0x123456789abcdef0;
```

```
x86  
movabs r10, 0x123456789abcdef0  
49 ba f0 de bc 9a 78 56 34 12
```

Code density

```
uint64_t v = 0x123456789abcdef0;
```

```
x86
movabs r10, 0x123456789abcdef0
49 ba f0 de bc 9a 78 56 34 12
```

```
MIPS
li      $2, 38141952
ori     $2, $2, 0x8acf
sll     $2, $2, 17
daddiu  $2, $2, 9903
sll     $2, $2, 18
ori     $2, $2, 0xdef0
46 02 02 3c cf 8a 42 34 78 14 02 00 af 26 42 64 b8 14 02 00 f0 de 42 34
```

Code density

```
uint64_t v = 0x123456789abcdef0;
```

```
x86
movabs r10, 0x123456789abcdef0
49 ba f0 de bc 9a 78 56 34 12
```

```
MIPS
li      $2, 38141952
ori     $2, $2, 0x8acf
dsll    $2, $2, 17
daddiu  $2, $2, 9903
dsll    $2, $2, 18
ori     $2, $2, 0xdef0
46 02 02 3c cf 8a 42 34 78 14 02 00 af 26 42 64 b8 14 02 00 f0 de 42 34
```

```
RISC-V
li      a5, 305418240
addi    a5, a5, 1657
li      a0, -1698897920
slli    a5, a5, 32
addi    a0, a0, -272
add     a0, a5, a0
12 34 57 b7 67 97 87 93 9a bc e5 37 02 07 97 93 ef 05 05 13 00 a7 85 33
```

Code density

```
uint64_t v = 0x123456789abcdef0;
```

```
x86
movabs r10, 0x123456789abcdef0
49 ba f0 de bc 9a 78 56 34 12
```

```
MIPS
li      $2, 38141952
ori     $2, $2, 0x8acf
dsll    $2, $2, 17
daddiu  $2, $2, 9903
dsll    $2, $2, 18
ori     $2, $2, 0xdef0
46 02 02 3c cf 8a 42 34 78 14 02 00 af 26 42 64 b8 14 02 00 f0 de 42 34
```

```
RISC-V
li      a5, 305418240
addi    a5, a5, 1657
li      a0, -1698897920
slli    a5, a5, 32
addi    a0, a0, -272
add     a0, a5, a0
12 34 57 b7 67 97 87 93 9a bc e5 37 02 07 97 93 ef 05 05 13 00 a7 85 33
```

```
SPARC
sethi    %hi(0x12345400), %g1
sethi    %hi(0x9ABCD00), %o0
or       %g1, 0x278, %g1
or       %o0, 0x2F0, %o0
sllx     %g1, 32, %g1
add      %g1, %o0, %o0
15 8d 04 03 37 af 26 11 78 62 10 82 f0 22 12 90 20 70 28 83 08 40 00 90
```

Code density

```
uint64_t v = 0x123456789abcdef0;
```

```
x86
movabs r10, 0x123456789abcdef0
49 ba f0 de bc 9a 78 56 34 12
```

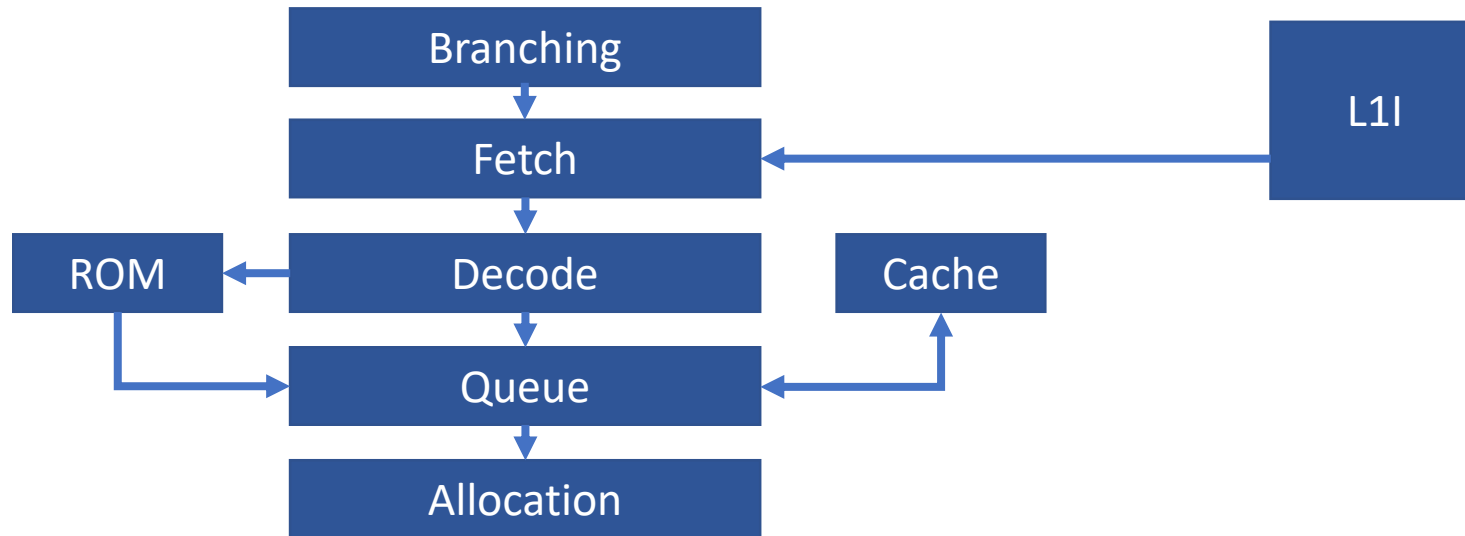
```
MIPS
li      $2, 38141952
ori     $2, $2, 0x8acf
dsll    $2, $2, 17
daddiu  $2, $2, 9903
dsll    $2, $2, 18
ori     $2, $2, 0xdef0
46 02 02 3c cf 8a 42 34 78 14 02 00 af 26 42 64 b8 14 02 00 f0 de 42 34
```

```
ARM
mov     x0, 0xdef0
movk    x0, 0x9abc, lsl 16
movk    x0, 0x5678, lsl 32
movk    x0, 0x1234, lsl 48
00 de 9b d2 80 57 b3 f2 00 cf ca f2 80 46 e2 f2
```

```
RISC-V
li      a5, 305418240
addi    a5, a5, 1657
li      a0, -1698897920
slli    a5, a5, 32
addi    a0, a0, -272
add     a0, a5, a0
12 34 57 b7 67 97 87 93 9a bc e5 37 02 07 97 93 ef 05 05 13 00 a7 85 33
```

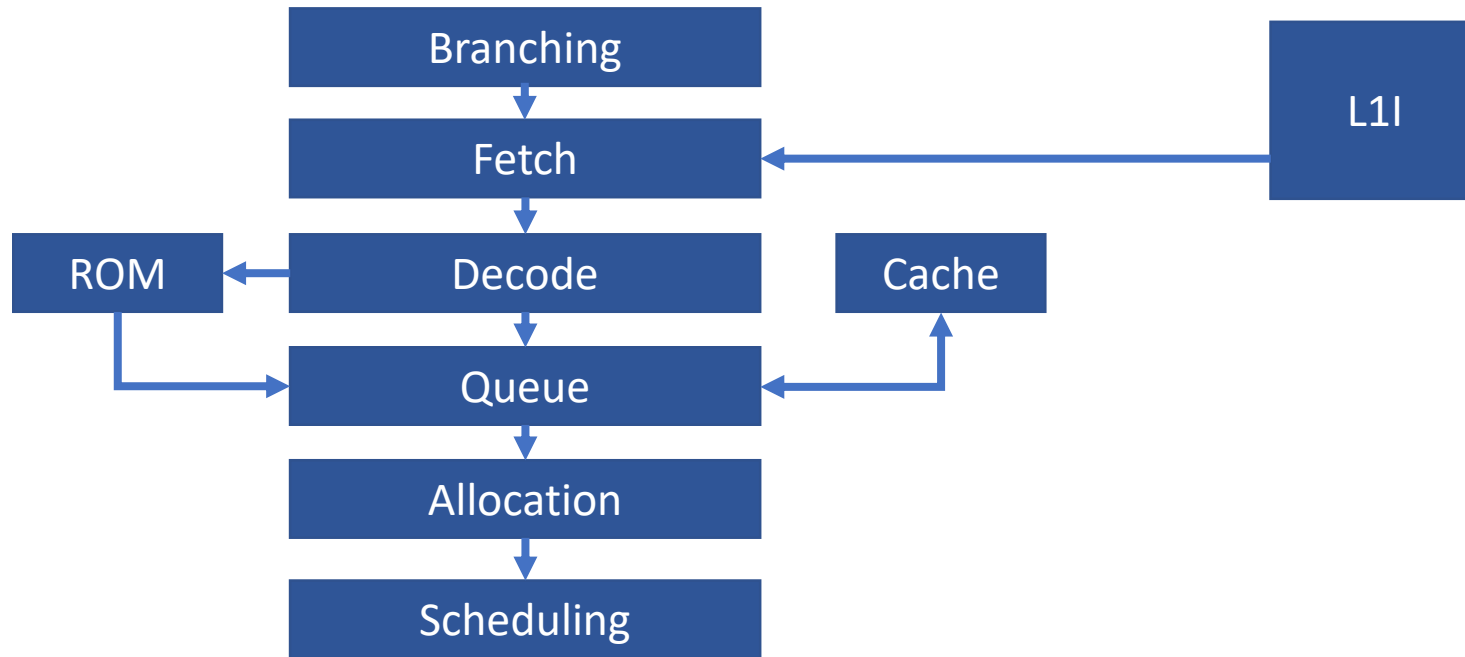
```
SPARC
sethi    %hi(0x12345400), %g1
sethi    %hi(0x9ABCD00), %o0
or       %g1, 0x278, %g1
or       %o0, 0x2F0, %o0
sllx     %g1, 32, %g1
add      %g1, %o0, %o0
15 8d 04 03 37 af 26 11 78 62 10 82 f0 22 12 90 20 70 28 83 08 40 00 90
```

Register renaming



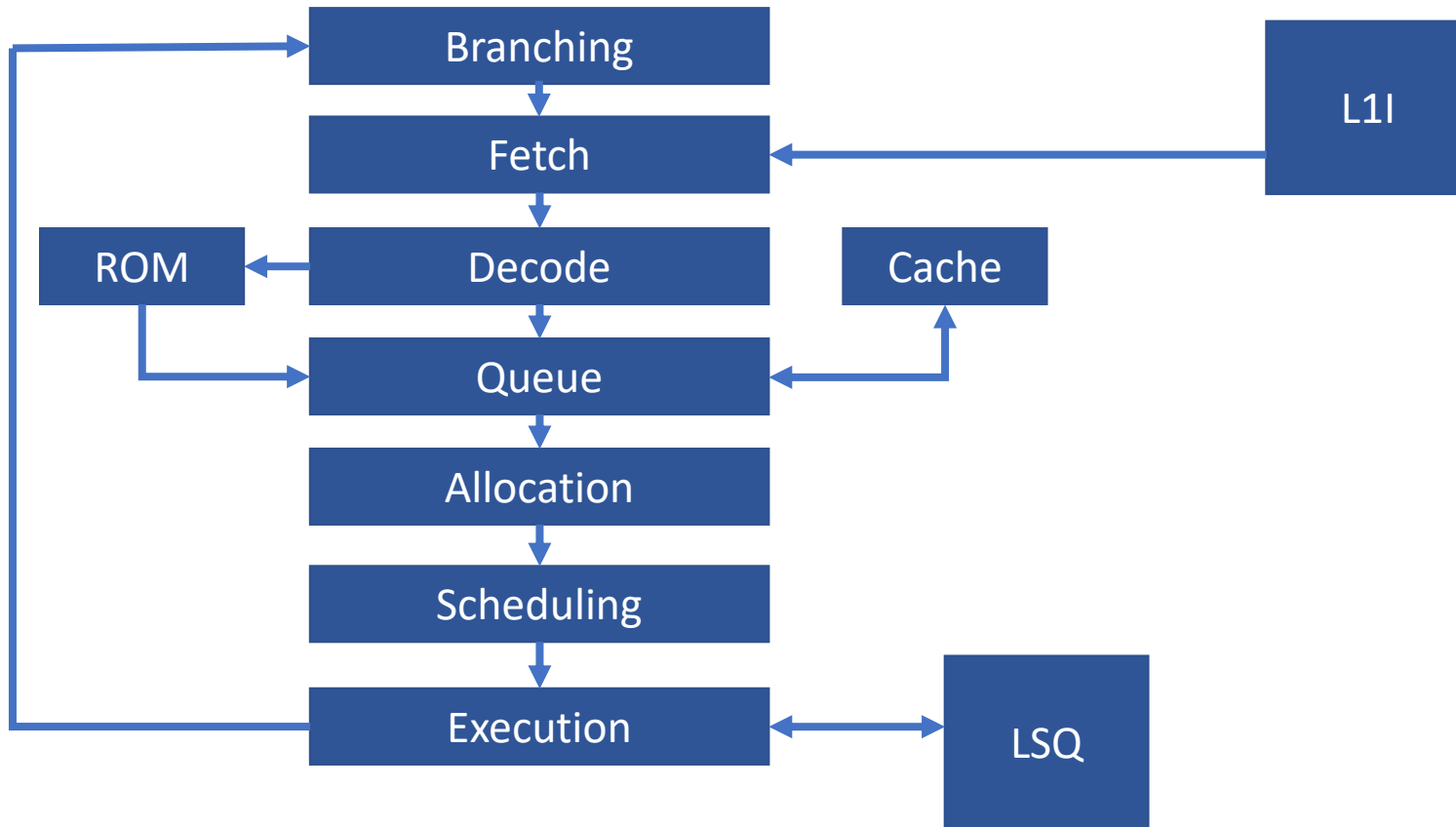
- ABI registers implement a SW contract
- They do not correspond to the actual execution
- Conversion of control flow to a variant of data flow
- Really complex
- Some operations end here
- Can a smart compiler help here?
- WLIV history

Scheduling



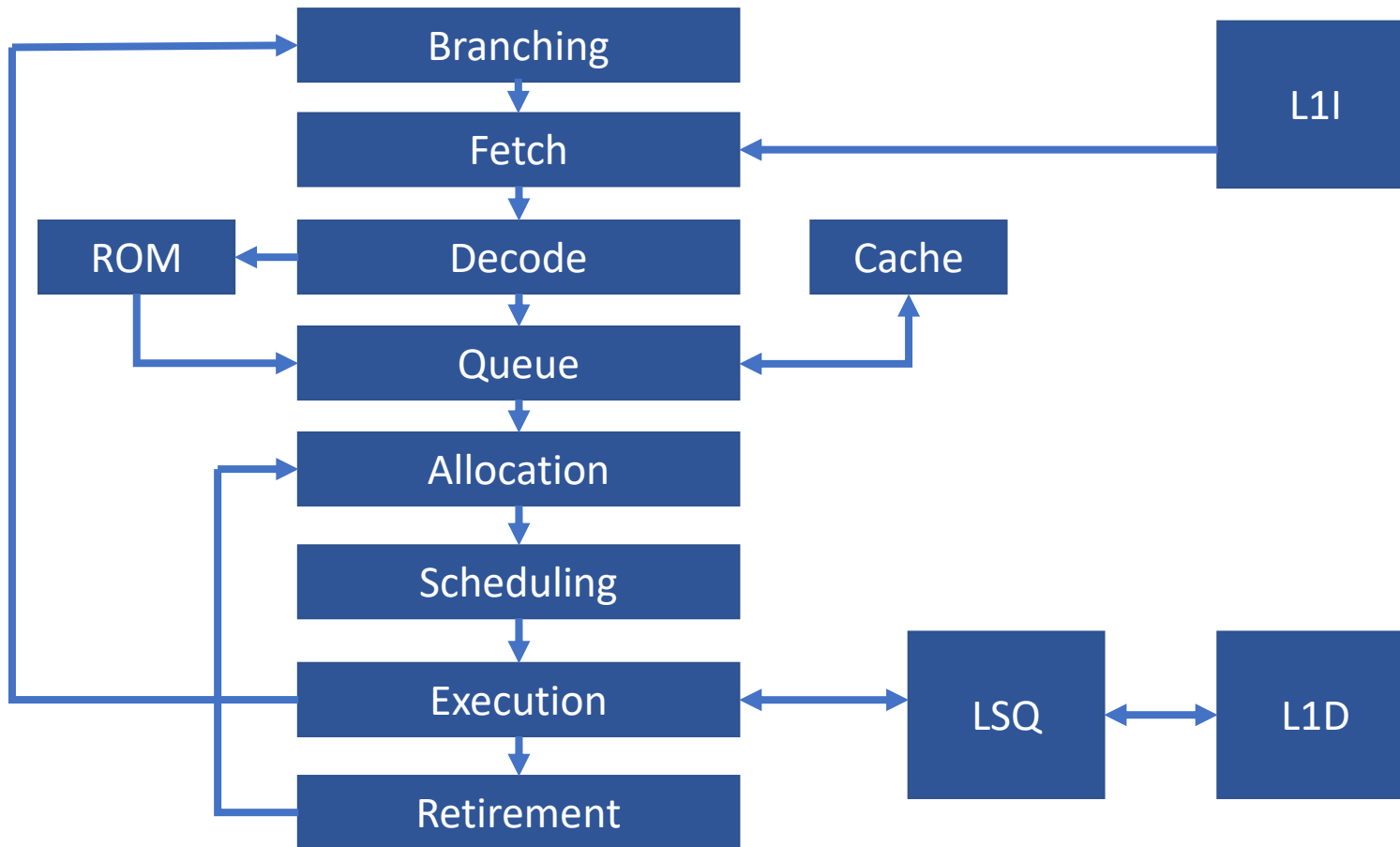
- Not all operations are equal
- Not all combinations of operations are equal

Execution



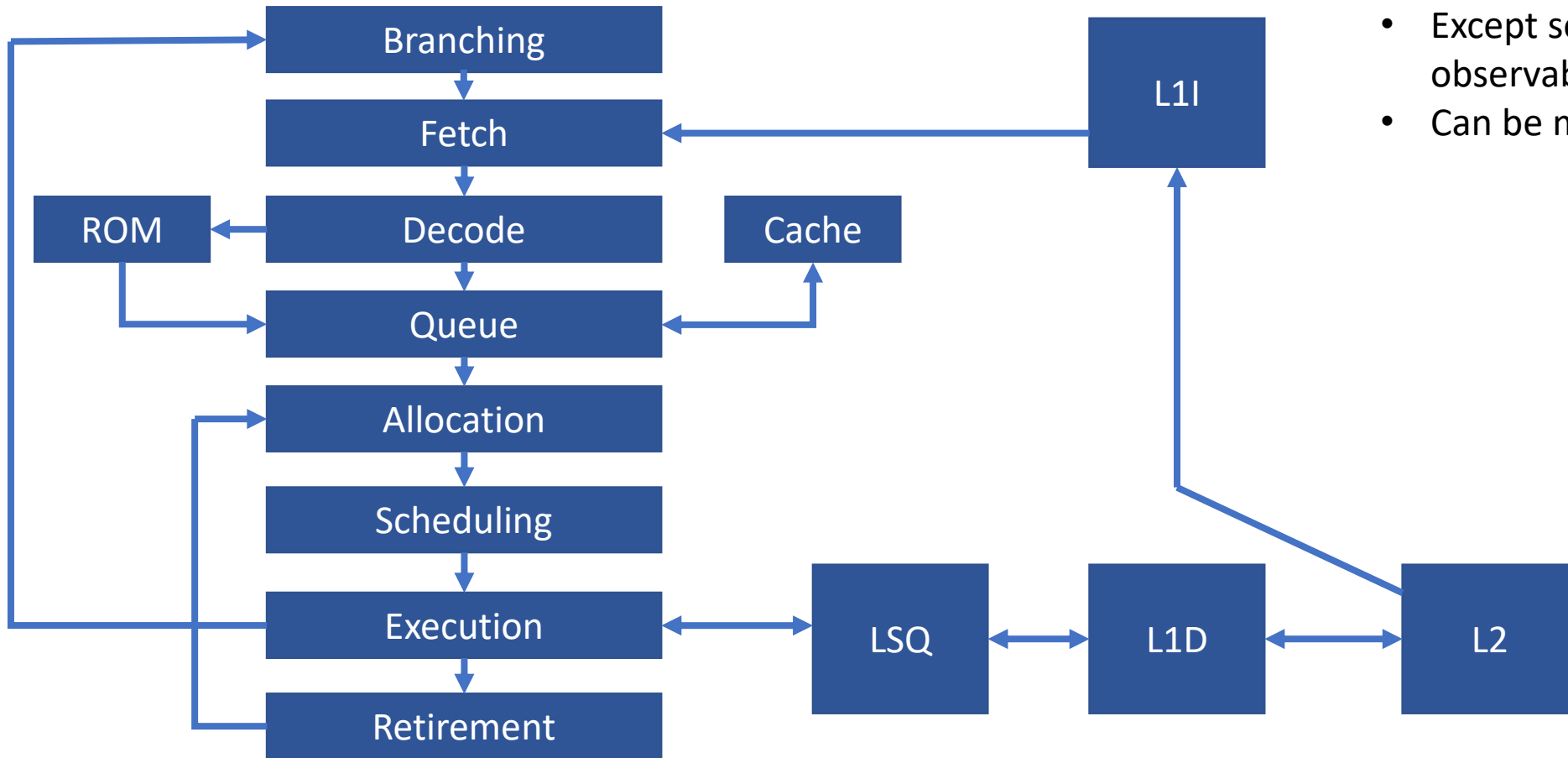
- Multiple specialized functional units
- Performs the actual (eventually) externally visible work
- Cycles
- Latency vs throughput

Retirement



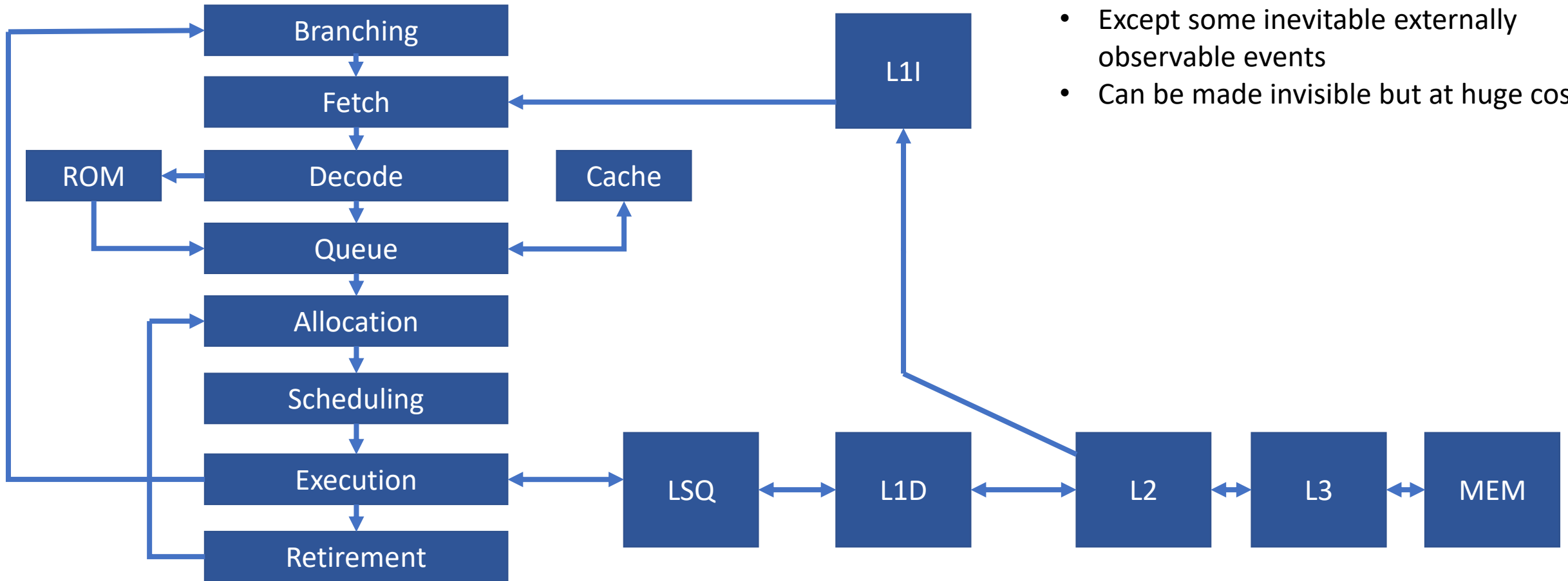
- All operations until now are speculative
- Except some inevitable externally observable events
- Can be made invisible but at huge cost

Retirement



- All operations until now are speculative
- Except some inevitable externally observable events
- Can be made invisible but at huge cost

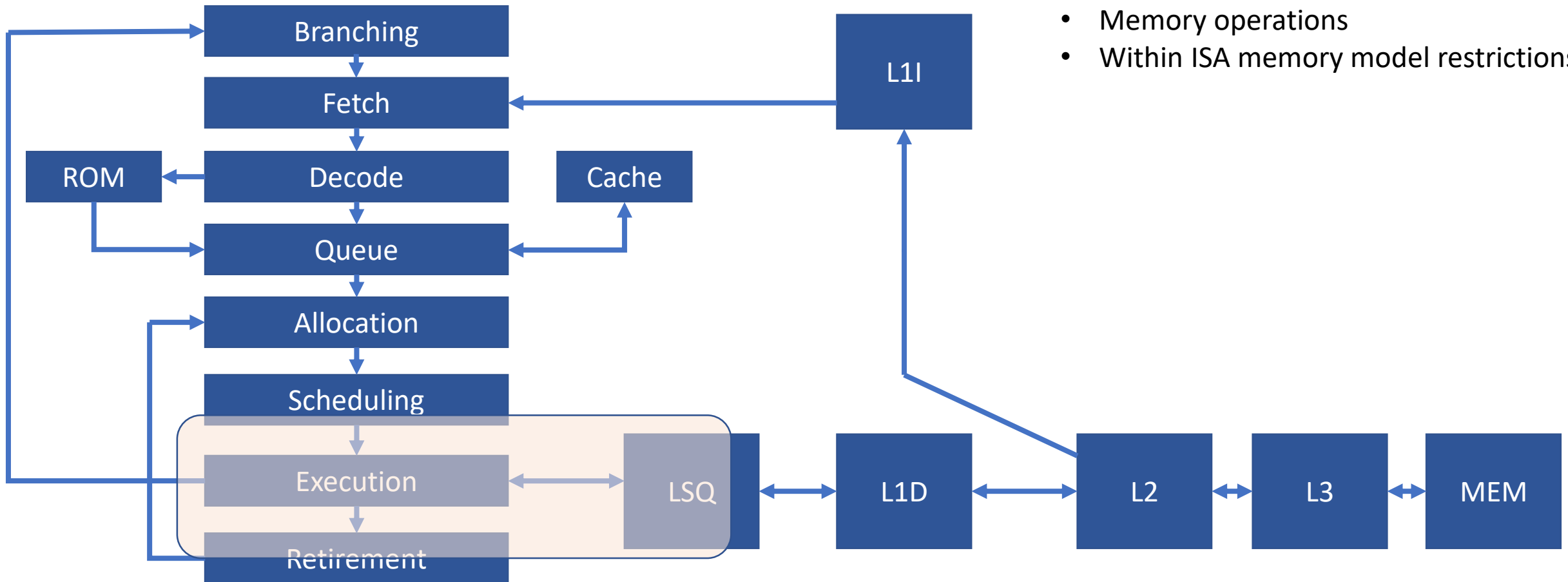
Retirement



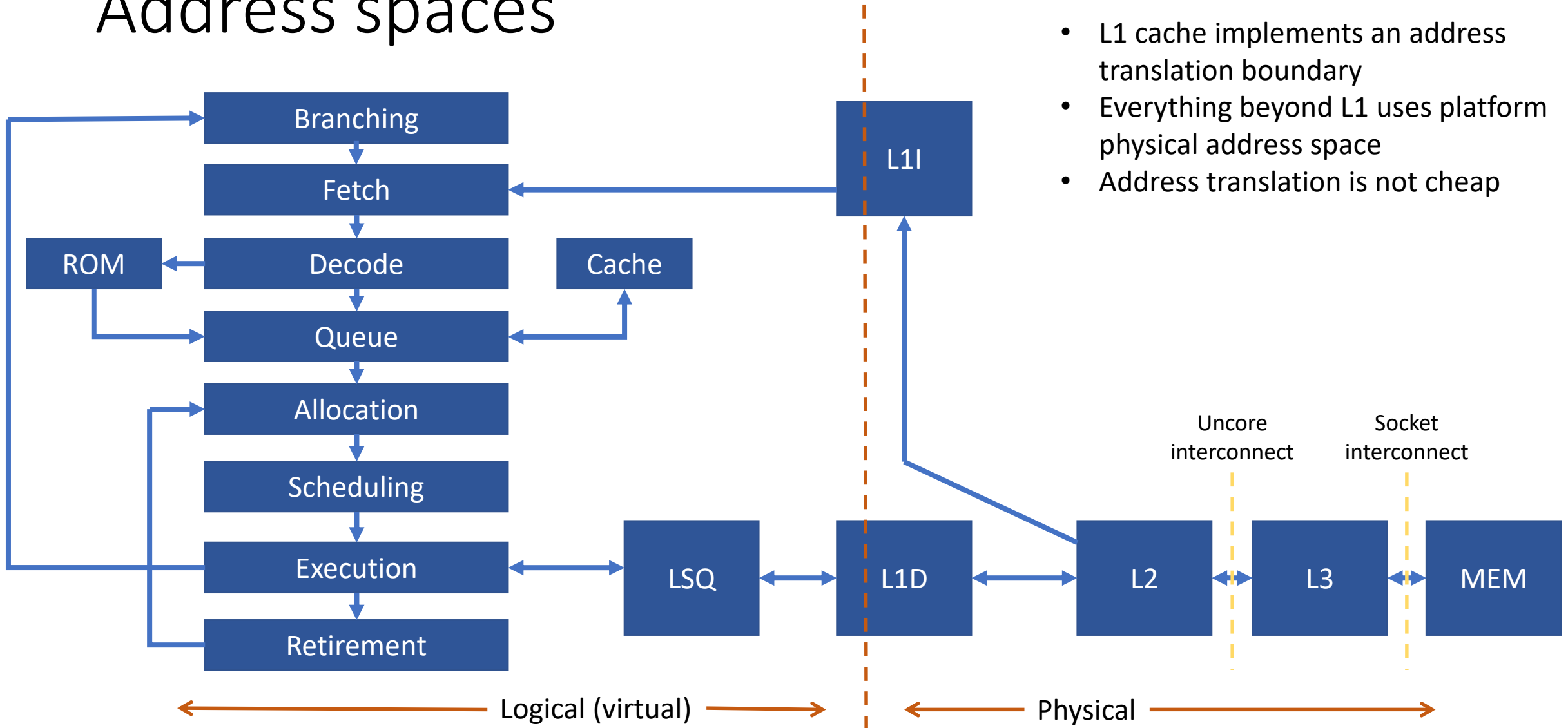
- All operations until now are speculative
- Except some inevitable externally observable events
- Can be made invisible but at huge cost

What precisely is out of order?

- Execution of operations
- With some exceptions
- Memory operations
- Within ISA memory model restrictions



Address spaces



- L1 cache implements an address translation boundary
- Everything beyond L1 uses platform physical address space
- Address translation is not cheap

memcpy !

- Why bother?
- Performance requirements
- Scalar, various vectors, ERMS, on-core and off-core accelerators
- Data layout: both software and hardware characteristics

Measuring performance

- All of this is very platform specific
- Core and uncore performance counters
- Granularity, resolution, value limits
- Not everything at once
- Microarchitectural microbenchmarks are fun
- Although nowhere near being easy
- And might not matter that much in a global scope
- Fleetwide benchmarks do matter though

Universal ISA

- Comparing x86, ARM, R-V, some historical others
- RISC vs CISC debate
- 32 → 31 → even less registers
- L/S vs R/M
- Flags
- Operand count
- Destructive destination
- Spilling strategy
- ISA as a contract

Vectorization?

- The logic of a scalar algorithm applied to a multitude of separate sets of data.
- Performance characterization in terms of latency and throughput.
- SIMD as a specific instantiation of vectorization approach.
- Practical and Everyday => relies on the compiler.

Vectorization in 20 minutes

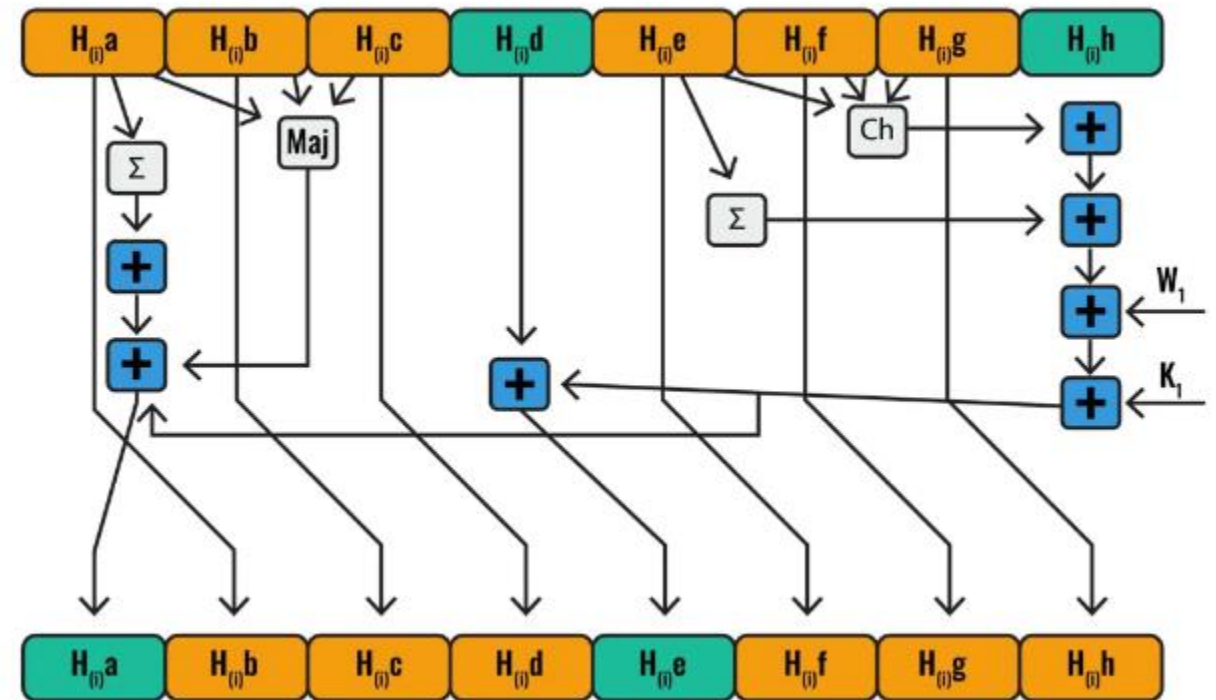
In order to vectorize scalar code successfully, horizontal operations are mandatory.
Control flow dependencies need to be translated to data flow dependencies.
Data structure layout should not conflict with native vector width and length.

- Controllable/maskable access to memory
- Assembling a vector register from scalar values (insert/extract)
- Mixing multiple vectors into one (blend)
- Swapping blocks and elements within a vector (permute, shuffle)
- Nonlinear and indexed memory access (scatter, gather)
- Conditional execution (predicates, masking)

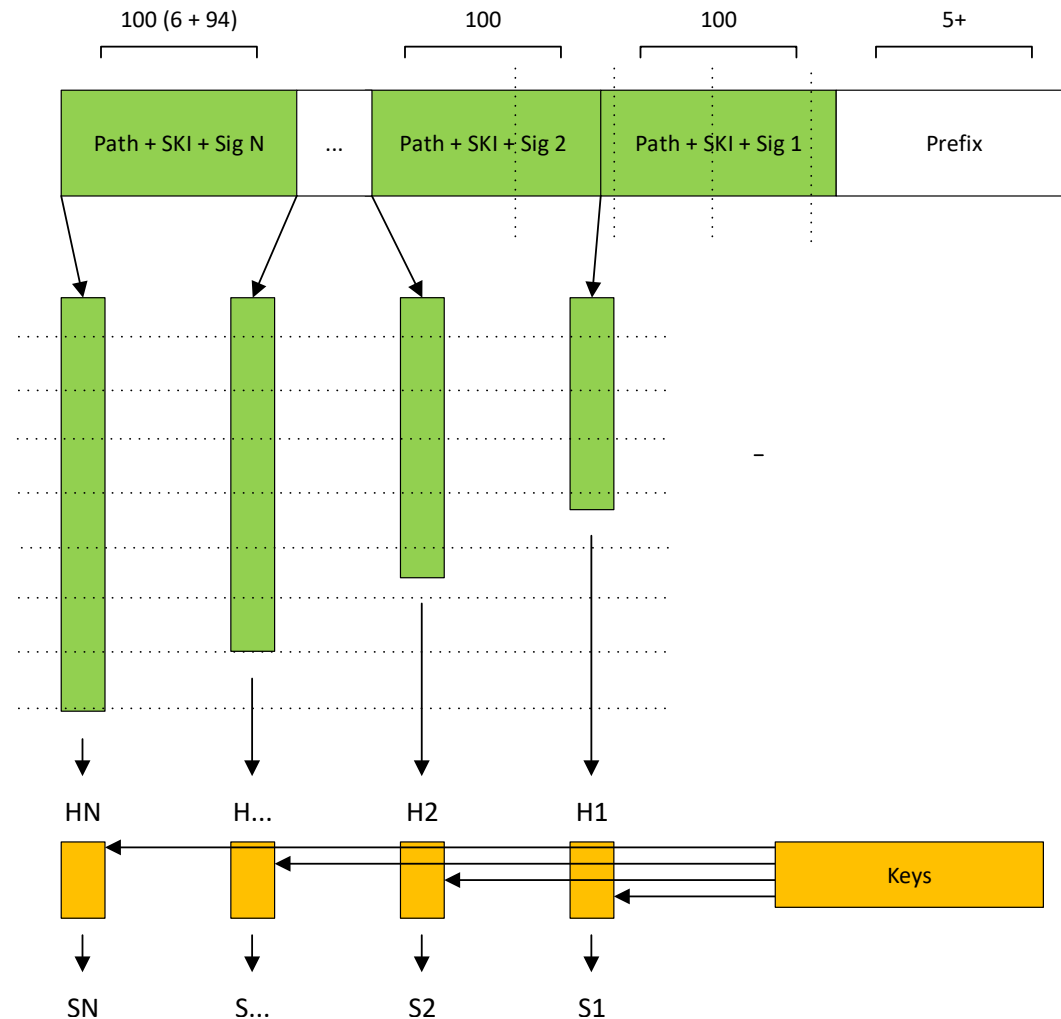
Vectorization Example – SHA2

- Block-based cryptographic hash function.
- Simple bitwise operations – many of them.
- Instruction set equivalence may not be assumed.
- Vertical vs horizontal data layout.

	Latency	Throughput
Scalar	1	1
Accelerated	4	2.2
Vectorized	0.85	15



Vectorized SHA2 and P-256 ECDSA



Linear code block operating on different data sets in parallel

Hash multiple blocks in parallel
Sign/verify multiple hashes/signatures in parallel

Vector lanes of fixed width

Gather operations place significant restrictions on data format

+20% latency results in +1500% throughput

Only if data structures allow!

Why bother?

Compiler is smart, one just needs to specify a correct command-line option, no?

```
for (int i = 0; i < m; i++) {  
    for (int j = 0; j < n; j++) {  
        for (int p = 0; p < k; p++) {  
            C(i, j) += A(i, p) * B(p, j);  
        }  
    }  
}
```

For vertical operations that is indeed not a complex task to do, and compilers perform just fine.

Why bother?

What if the level of triviality is reduced a little?

- Control flow dependencies.
- Different lane widths and parameters
- Branching
- Iterations of different length

```
missed: not vectorized: complicated access pattern.  
missed: not vectorized: no vectype for stmt: sum[0] = { 0, 0, 0, 0 };  
missed: couldn't vectorize loop
```

```
uint32_t CityHash32(const char *s, size_t len) {  
  
    if (len <= 24) {  
        return len <= 12 ? (len <= 4 ?  
            Hash32Len0to4(s, len) :  
            Hash32Len5to12(s, len)) :  
            Hash32Len13to24(s, len);  
    }  
  
    h ^= a0;  
    h = h * 5 + 0xe6546b64;  
    h ^= a2;  
  
    do {  
        h ^= a0;  
        h = Rotate32(h, 18);  
        h = h * 5 + 0xe6546b64;  
        f += a1;  
        s += 20;  
    } while (--iters != 0);  
  
    h = h * 5 + 0xe6546b64;  
    h = Rotate32(h, 17) * c1;  
    h = Rotate32(h + f, 19);  
    h = h * 5 + 0xe6546b64;  
    h = Rotate32(h, 17) * c1;  
  
    return h;  
}
```

Historical Perspective

MMX (1996)	8 64-bit integer registers 8/16/32/64, saturation, two-operand.
SSE (1999):	8 128-bit int and fp registers, three-operand, some domain-specific accelerations.
AVX (2011):	16 256-bit fp-only registers, 128-bit lanes, 32/64-bit elements.
AVX2 (2013):	Integer AVX version, horizontal operations, gather, 8/16/32/64-bit elements.
LRBNI (2009):	32 x 512-bit data and 8 x 16-bit predicate registers, i32, f32/f64, gather and scatter, flexible bit manipulation.
IMCI (2010):	32 + 8 registers, cache management, fp focus.
AVX-512 (2015)	IMCI backport to AVX2, 32 + 8 registers, int and fp focus.

Vectorize what?

- Historically the domain of HPC
- Differential equation systems and CFD simulations are important
- TLS termination is important too
- An open question remains – what is the ratio of cores doing the former and the latter?

Discussion